Programming Assignment 2: TCP Header Simulation in Client-Server Communication

Introduction

In this assignment, you will continue working with TCP socket programming to build a simplified **client-server model** that simulates key features of TCP headers. This project focuses on **creating, sending, parsing, and responding to structured messages** that contain a custom-defined header.

You will design a fixed-format message structure where the client includes a **custom TCP-like header** in each message. The server will parse this header and respond based on the values of specific fields, mimicking how real TCP stacks process control bits and metadata.

This assignment reinforces **low-level protocol concepts** and provides hands-on experience with:

- TCP/IP socket programming
- Structuring and parsing binary data
- Conditional server behavior based on message headers

Unlike Assignment 1, this project is limited to a **single-client model** and does **not** require encryption.

Guidelines

Project Requirements

1. GitHub Repository

- Your project must be stored in a **private GitHub repository**.
- Include a **clear commit history** showing progressive development.

2. TCP Server Implementation

- Listens on a specific port for a single incoming connection.
- Receives structured messages from the client.
- Parses a **custom TCP-like header** from the message.
- Responds based on the header fields (see "Custom Header Specification" below).
- Logs all received headers and server responses.

3. TCP Client Implementation

- Connects to the server.
- Constructs and sends messages with a **custom TCP header** followed by a simple payload.
- Displays the server's response based on the header flags.

4. Custom TCP Header Specification

Your program must define a custom TCP-style header with the following fixed fields:

Field	Size	Description
Source Port	2 bytes	Arbitrary client-side port number
Dest Port	2 bytes	Server's listening port
Sequence No	4 bytes	Sequence number of the message
ACK Flag	1 byte	0 or 1 (acknowledgment)
SYN Flag	1 byte	0 or 1 (synchronize/start handshake)
FIN Flag	1 byte	0 or 1 (finish/terminate connection)
Payload Size	2 bytes	Length of payload following header

Total Header Size: 13 bytes
Message Structure = **[13-byte header] + [payload]**

Use `struct` or an equivalent method to build and parse the binary message.

5. Server Response Logic

- If **SYN** is 1, server responds: `"SYN received – connection initiated"`
- If **ACK** is 1, server responds: `"ACK received – message acknowledged"`
- If **FIN** is 1, server responds: `"FIN received – connection closing"`
- Otherwise, server responds: `"Data received – payload length: X"`

6. Error Handling

- Graceful handling of malformed headers or disconnections.
- Appropriate logging of unexpected input.

7. Multi-File Project & Compilation

- Separate files for client and server.
- Use a **Makefile** (see below).

Project Structure and Submission

Makefile Requirements

Include a `Makefile` with the following targets:

- `make build` – Compile the project.
- `make run-server` – Run the server.
- `make run-client` – Run the client.
- `make clean` – Remove compiled artifacts.

README.md

Your `README.md` should include:

- How to build and run both programs.
- Required libraries (e.g., `struct`, `socket`, etc.).
- Explanation of how the message header works.

Design Explanation Document

Include `design_explanation.md` or `.pdf` describing:

- The format and structure of the custom header.
- How parsing and response logic works on the server.
- Reasoning for any error-handling or design decisions.

Grading Rubric

Criteria	Points	Description
Simplicity & Cleanliness of Code	25	Modular, readable, and logically structured implementation.
Documentation & Explanation	20	Clear and complete `README` and design document.
Proper Use of GitHub & Makefile	20	Good Git practices, correct Makefile structure and use.
Custom Header Implementation	15	Correct creation, parsing, and handling of the custom TCP-style header.
Successful Execution	20	Handles specified header flags correctly and logs output as required.
Total	**100**	

Bonus (+5 points): Implement checksum verification for the header using a simple sum or hash.

Additional Notes

- You may use **C, C++, Python, or Other Language**, but clearly document all dependencies.
- Ensure that your server **does not crash** from malformed messages or disconnects.

Deadline: April 13, 2025, midnight